

Proposta do Projeto Desenvolvimento Mobile e IoT

1. Visão Geral do Projeto

Alunos: Gaisani Calle, Dimas Vianna

Nome do Aplicativo: Lumees Yapplant

Objetivo

Desenvolver um sistema integrado de IoT, aplicação mobile e Inteligência Artificial para monitoramento residencial de plantas, capaz de coletar dados ambientais em tempo real e classificar o estado de saúde da planta através de um modelo preditivo.

Plataforma

- Android
- Web

Tecnologia

- **Front-end:** Flutter
- **Linguagem:** Dart
- **Back-end:** Python
- **Banco de Dados:** Firebase NoSQL

2. Requisitos Funcionais

- **RF01 - Coleta de Dados**
 - O hardware deve ler periodicamente os níveis de umidade do solo, luminosidade ambiente e temperatura/umidade do ar.
- **RF02 - Envio de Dados**
 - O dispositivo deve transmitir as leituras coletadas para o servidor backend via Wi-Fi.
- **RF03 - Armazenamento**
 - O sistema deve salvar o histórico de leituras dos sensores em um banco de dados.
- **RF04 - Classificação de Saúde (IA)**

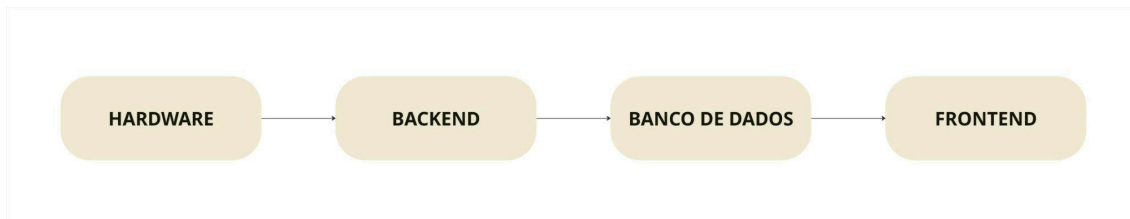
- O modelo em Python deve analisar o conjunto de dados dos sensores e classificar o estado da planta em categorias (ex: Saudável, Risco de Desidratação, Risco de Fungos/Excesso de Água, Falta de Luz).
 - **RF05 - API de Integração**
 - O backend deve expor endpoints para receber os dados do hardware e fornecer o status atualizado + diagnóstico da IA para a interface mobile.
 - **RF06 - Exibição do Avatar**
 - A interface deve renderizar o visual do Tamagotchi refletindo diretamente a classificação gerada pela IA (ex: suando se estiver muito quente, triste se estiver seco).
 - **RF07 - Painel de Atributos**
 - O usuário deve conseguir visualizar as barras de status da planta (Sede, Luz, Conforto Térmico).
-

3. Requisitos Não Funcionais

- **RNF01 - Tempo de Resposta (Atualização)**
 - A interface deve atualizar o estado do Tamagotchi em até 5 segundos após o recebimento de uma nova leitura de dados enviada pelo hardware.
 - **RNF02 - Baixo Consumo de Dados**
 - O intervalo de envio de dados do hardware para o backend deve ser otimizado (a cada 10 minutos) para evitar sobrecarga no servidor e economizar energia.
 - **RNF03 - Portabilidade da Interface**
 - A aplicação de interface deve ser responsiva, garantindo o funcionamento correto e boa usabilidade em telas de smartphones (Android) e em telas de computador.
 - **RNF04 - Simplicidade do Modelo de IA**
 - O algoritmo de Machine Learning utilizado deve ser leve o suficiente para rodar o treinamento e a predição no servidor backend sem exigir hardware de alta performance (GPUs).
 - **RNF05 - Confiabilidade da Conexão**
 - O firmware do hardware deve ser capaz de tentar reconectar automaticamente à rede Wi-Fi caso a conexão caia, sem travar a execução do código.
-

4. Arquitetura do Sistema

Arquitetura

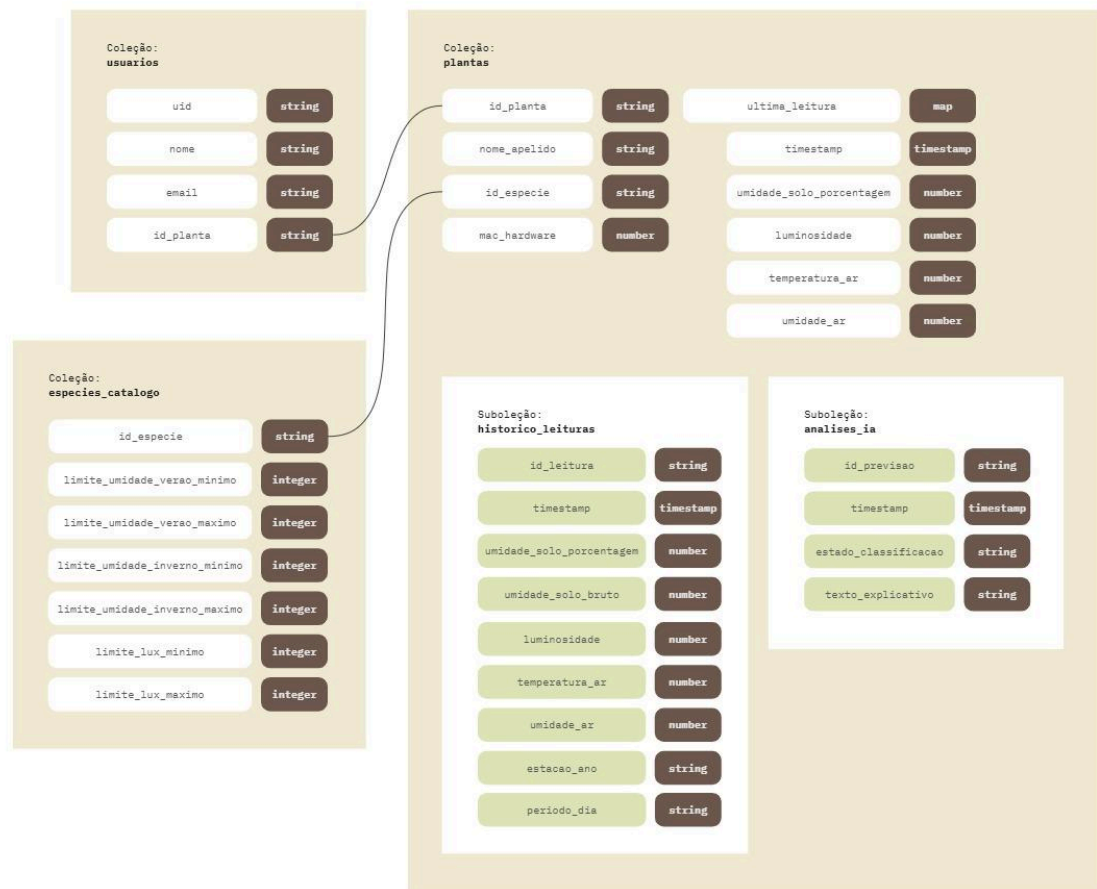


Camadas

O sistema é estruturado em quatro camadas principais que trabalham de forma integrada: o Hardware (sensores), o Backend (servidor e inteligência artificial), o Banco de Dados (armazenamento) e o Frontend (aplicativo móvel). Essa divisão garante que cada parte do projeto tenha uma função específica e isolada, facilitando a manutenção e a comunicação dos dados.

1. **Camada de Captura (Hardware):** Composta pelo microcontrolador ESP32 e sensores físicos. Esta camada é responsável por medir as variáveis do ambiente (como a umidade do solo e a temperatura) e enviar esses dados brutos para o Backend através de requisições na rede.
 2. **Camada de Processamento (Backend e IA):** Desenvolvida em Python com o framework FastAPI. É o "cérebro" do sistema. O Backend recebe os dados enviados pelo Hardware, analisa e trata os dados brutos, classifica um estado de saúde da planta (ex: se está saudável, com sede ou encharcada) e envia para o Banco de Dados.
 3. **Camada de Persistência (Banco de Dados):** Gerenciada pelo Firebase. Esta camada armazena de forma estruturada todo o histórico de leituras dos sensores, os diagnósticos gerados pela Inteligência Artificial e as informações de cadastro dos usuários, servindo como a fonte central de dados do projeto.
 4. **Camada de Visualização (Frontend Mobile):** Desenvolvida em Flutter. É a interface gráfica (aplicativo mobile) com a qual o usuário interage. Ela consome os dados armazenados no Banco de Dados e as respostas em tempo real da API para exibir gráficos, alertas de saúde da planta e painéis de controle de forma visual e intuitiva.
-

5. Modelagem do Banco de Dados



6. Relacionamento Entre Tabelas

- 1 Usuário tem 1 Planta: **usuarios 1:1 plantas**
- 1 Planta tem 1 Usuário: **plantas 1:1 usuarios**
- 1 Planta tem Várias Leituras: **plantas 1:n historico_leituras**
- 1 Planta tem Várias Análises de IA: **plantas 1:n analises_ia**
- 1 Espécie tem Várias Plantas: **catalogo_especie n:1 plantas**

7. Fluxo do Aplicativo

Fluxo Principal

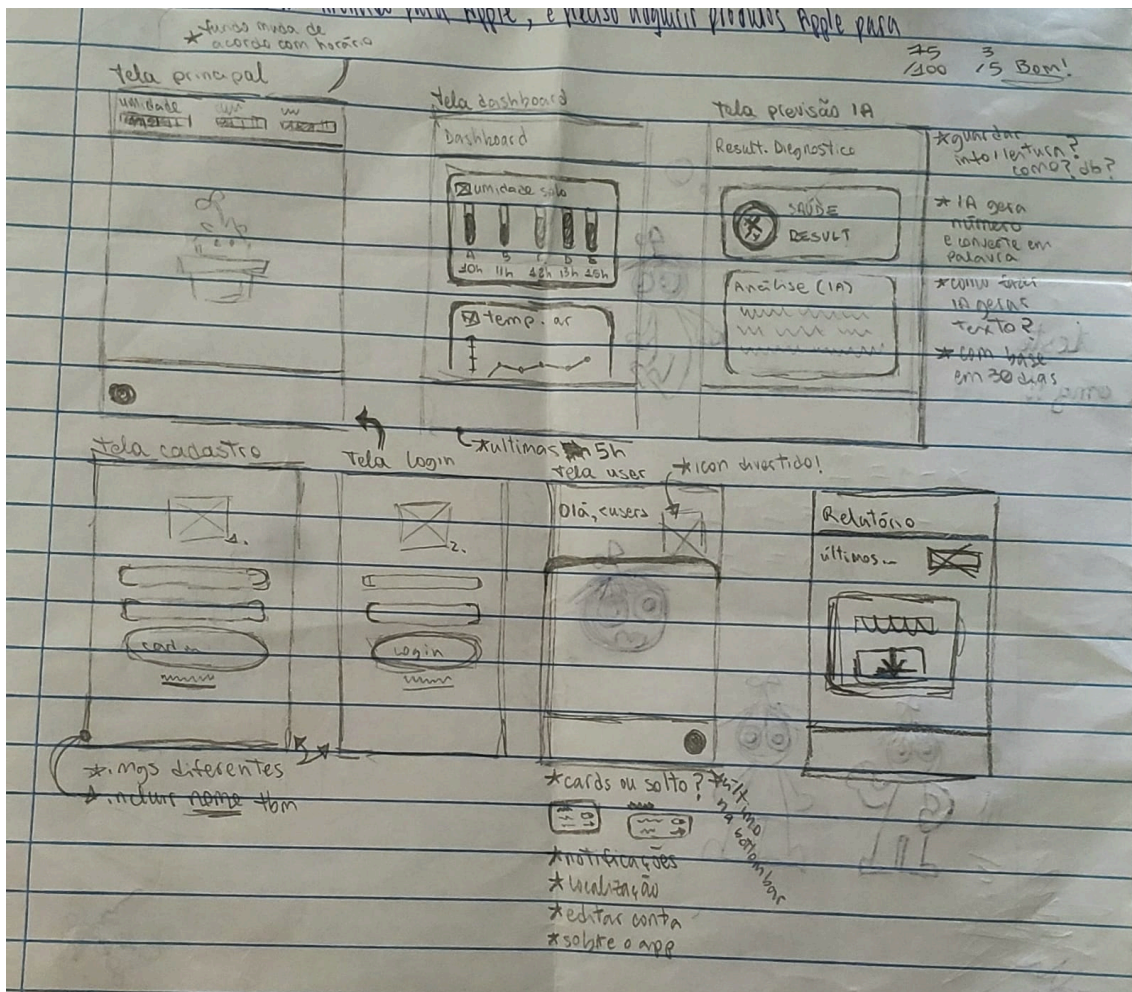
O fluxo de navegação do aplicativo foi projetado para ser intuitivo, garantindo que o usuário acesse as informações de saúde da sua planta com o menor número de cliques possível. A jornada dentro do ecossistema mobile foi estruturada nas seguintes etapas sequenciais:

1. **Boas-Vindas:** O usuário interage inicialmente com a Tela Inicial, que apresenta uma mensagem de boas-vindas e o botão "Começar".
2. **Autenticação:** Ao avançar, o sistema direciona o usuário para a tela de autenticação, permitindo a escolha entre Criar Conta (para o primeiro acesso) ou o Login (para quem já possui uma conta ativa).
3. **Tela Principal Dinâmica:** Após a validação do acesso, o usuário entra na Tela Principal. Esta tela possui uma lógica condicional inteligente baseada no banco de dados:
 - **Sem planta cadastrada:** A interface esconde o avatar da planta e exibe um cartão (\textit{card}) de destaque solicitando o cadastro do primeiro dispositivo e da espécie.
 - **Com planta cadastrada:** O cartão de aviso desaparece, dando lugar ao avatar gráfico da planta, que muda de expressão de acordo com o estado de saúde atual.
4. **Configuração de Rede (Wi-Fi):** Para que os dados comecem a ser transmitidos, o usuário precisa conectar o hardware à rede local. Caso seja a primeira vez, o aplicativo oferece instruções claras na tela de configurações para realizar esse vínculo; se a rede já estiver configurada, o sistema inicia a comunicação automaticamente.

Uma vez estabelecida a conexão e o sensor inserido no solo, o usuário ganha acesso livre para navegar pelas cinco telas principais de gerenciamento do aplicativo:

- **Monitoramento em Tempo Real (Tela Principal):** Exibe os dados instantâneos enviados pelos sensores e as reações visuais da planta.
 - **Análise Histórica (Tela Dashboard):** Apresenta gráficos contendo as leituras coletadas nas últimas 5 horas, permitindo acompanhar a evolução recente do ambiente.
 - **Diagnóstico Avançado (Tela Diagnóstico IA):** Espaço onde o usuário pode solicitar manualmente uma análise aprofundada para o modelo de Inteligência Artificial (Lee).
 - **Exportação de Dados (Tela Relatório):** Permite gerar e baixar um arquivo em formato CSV com o histórico completo de medições dos últimos 15 dias.
 - **Gerenciamento do Perfil (Tela de Usuário):** Painel onde é possível visualizar e editar dados pessoais (nome, e-mail e senha), gerenciar permissões de privacidade do sistema (como o uso da localização) e consultar o manual de instruções de configuração do Wi-Fi.
-

8. Wireframe Simplificado



9. Detalhamento das Telas Desenvolvidas

9.1. Telas de Acesso (Inicial, Login e Cadastro)

A jornada se inicia na **Tela Inicial**, uma interface limpa com foco em conversão que apresenta a marca do projeto e direciona o usuário ao fluxo de entrada. A **Tela de Login** e a **Tela de Cadastro** possuem campos validados para inserção de credenciais (e-mail e senha). O cadastro armazena as informações básicas no Firebase para criar o perfil único do usuário (uid).

9.2. Tela Principal (Monitoramento Real-Time)

Esta é a tela de maior permanência do usuário. Ela consome dados diretamente do banco de dados para renderizar seu conteúdo de forma dinâmica. Caso o sistema identifique que o usuário não possui um dispositivo atrelado, exibe-se um cartão de ação (`\textit{card}`) centralizado para o cadastro. Havendo um dispositivo ativo, a tela exibe os dados de leitura

instantânea dos sensores e o avatar gráfico da planta, cujo estado visual (expressão e animação) responde diretamente aos dados de saúde calculados pelo Backend.

9.3. Tela Dashboard (Gráficos Históricos)

Voltada para a análise de tendências, esta tela organiza as leituras brutas capturadas pelos sensores em uma linha do tempo retroativa de até 5 horas. Os dados são plotados em gráficos de linha dinâmicos, permitindo que o usuário visualize de forma simples se a umidade e a temperatura do ambiente estão se mantendo estáveis ou se apresentam picos de anomalia.

9.4. Tela de Diagnóstico da Inteligência Artificial

Esta interface funciona como um painel de consulta avançada. Nela, o usuário encontra um botão de ação rápida para disparar uma requisição de análise. Ao ser acionada, a tela exibe o veredito do modelo KNN (Lee) de forma destacada (ex: "Crítico", "Saudável") acompanhado do bloco de texto explicativo gerado pela API.

9.5. Tela de Relatório (Exportação CSV)

Uma tela utilitária projetada para usuários que necessitam de um registros mais profundos. Ela exibe um botão para exportar e baixar uma tabela estruturada em formato CSV contendo todas as medições coletadas nos últimos 15 dias, facilitando o compartilhamento e a leitura externa dos dados.

9.6. Tela de Configurações e Usuário

Interface administrativa dividida em três blocos principais de controle:

- **Dados do Perfil:** Permite a edição segura de nome, e-mail e redefinição de senha.
 - **Privacidade e Permissões:** Chaves de ativação de recursos do sistema, como a permissão para uso da localização do dispositivo mobile.
 - **Suporte Técnico (Wi-Fi):** Mini guia com instruções para que o usuário saiba como parear o hardware do sensor à rede sem fio local.
-

10. Engenharia de Software e Backend

O microsserviço de backend do projeto *Lumees Yapplant* foi desenvolvido utilizando o *framework* assíncrono FastAPI, escolhido por sua alta performance, validação nativa de dados baseada em tipos complexos e geração automática de documentação. A arquitetura foi desenhada seguindo o princípio da modularidade, segregando as responsabilidades de negócio em roteadores específicos (`\textit{Routers}`) para garantir a escalabilidade e a manutenibilidade do código-fonte.

10.1. Documentação dos Endpoints (Rotas da API)

A comunicação com o backend é estruturada sob o prefixo de versão `/lumees-api/v1`. A validação de contratos e a tipagem de entrada dos dados de requisições utilizam classes de dados estruturadas através da biblioteca Pydantic.

As rotas disponíveis na aplicação são:

- `POST /lumees-api/v1/ia/analise`: Recebe o ID da planta, a espécie e a estação do ano atuais. Processa os dados no modelo de inteligência artificial correspondente e retorna a classificação de saúde da planta.
- `POST /lumees-api/v1/hardware/coleta`: Ponto de entrada para o dispositivo ESP32. Recebe o endereço MAC do hardware e as leituras dos sensores (umidade do solo, luminosidade, temperatura e umidade do ar).
- `GET /lumees-api/v1/plantas/{id_planta}/exportar-csv`: Recebe o identificador de uma planta na URL para gerar o histórico de leituras em formato de planilha.

10.2. Integração Backend-Firebase

O backend integra-se com os serviços do Firebase por meio do SDK oficial `firebase\_admin`, operando em duas frentes: banco de dados NoSQL Cloud Firestore e armazenamento de arquivos Cloud Storage.

No Cloud Firestore, as operações realizam o gerenciamento dos dados do aplicativo:

- **Vínculo de Hardware:** A rota de coleta busca no banco qual planta possui o `mac\_hardware` enviado pelo ESP32 para descobrir o `id\_planta` correto.
- **Histórico de Sensores:** Cada nova leitura do hardware é salva na subcoleção `historico\_leituras` com um timestamp em formato UTC.

O campo `ultima_leitura` na raiz do documento também é atualizado para atualizar a interface do usuário.

- **Busca Retroativa:** Para executar a IA, o backend faz uma busca no histórico da planta para recuperar o registro de umidade do solo de exatos 7 dias atrás. Se não houver dados, o sistema adota a umidade atual como padrão de segurança.
- **Resultados da IA:** O diagnóstico gerado pelo modelo é armazenado no histórico de análises da planta e atualiza o status em tempo real no documento principal.

No Cloud Storage, backend gerencia o fluxo de geração e exportação de relatórios da seguinte forma:

1. Busca no Firestore as leituras da planta referentes aos últimos 15 dias corridos.
 2. Formata esses dados no padrão CSV (separados por ponto e vírgula) utilizando buffers de texto diretamente na memória RAM, evitando gravação de arquivos temporários no servidor.
 3. Envia o arquivo de texto gerado para uma pasta no Cloud Storage sob o caminho `relatorios/historico_{id_planta}.csv`.
 4. Cria uma URL assinada (*Signed URL v4*) temporária e segura, com validade restrita de 1 hora e método GET, devolvendo esse link para o aplicativo realizar o download do arquivo.
-